

QA-L01 DIAGNÓSTICO INICIAL DEL PROYECTO

Código: QA-L01

Duración: 4 horas

Competencia: 38369 - Controlar la calidad del servicio de software de acuerdo con los estándares técnicos.

Resultado de Aprendizaje: 593146 - Incorporar actividades de aseguramiento de la calidad del software de acuerdo con estándares de la industria.

Objetivo: Realizar un diagnóstico técnico del proyecto formativo con el fin de identificar el estado actual del software, los riesgos existentes, el nivel de avance y las acciones necesarias para iniciar el proceso de aseguramiento de la calidad.

Resultados Esperados

Al finalizar el laboratorio cada equipo deberá:

- Conocer el estado real del proyecto.
- Identificar fortalezas y debilidades.
- Detectar riesgos.
- Identificar deuda técnica.
- Elaborar un informe diagnóstico.
- Definir prioridades para las pruebas.

Contexto

La empresa de desarrollo ha solicitado al equipo de QA iniciar el proceso de aseguramiento de la calidad.

Sin embargo, antes de construir el Plan Maestro de Pruebas, es necesario conocer el estado real del software.

Como nuevo equipo QA deberán realizar una auditoría inicial del proyecto.

Contextualización del proyecto

Realizar la contextualización del proyecto siguiendo los siguientes ítems:

Nombre del proyecto

Sistema de Gestión de Inventario y Ventas para Asadero La 75.

Problema que resuelve

El asadero realiza el control de inventario, compras y ventas de forma manual, lo que ocasiona errores en el stock, retrasos en la atención y poca confiabilidad en la información.

Objetivo

Desarrollar un sistema web y móvil que permita administrar usuarios, productos, categorías, proveedores, compras, ventas e inventario, facilitando la toma de decisiones y mejorando el control del negocio.

Usuarios

- Administrador
- Vendedor
- Encargado de inventario/compras

Arquitectura

Arquitectura cliente-servidor desacoplada mediante API REST.

Tecnologías utilizadas

- Laravel 12
- React
- Android Studio (Kotlin)
- MySQL (MariaDB)
- Laravel Sanctum
- Git y GitHub
- Vite

Sección 1. Organización del Repositorio GitHub

Criterio	Sí	No	Observaciones
Existe organización en GitHub	x		Repositorio organizado.
El nombre de los repositorios es consistente	x		Sigue el nombre del proyecto.
Existe README.md	x		Existe documentación básica.
Existe licencia		x	No se evidencia licencia.
Existe .gitignore	x		Configurado correctamente.
Se utilizan ramas (main, develop, feature)	x		main, develop y ramas feature.
Existen Pull Request	x		Utilizados durante el desarrollo.
Se utilizan Issues	x		Se utilizan ocasionalmente.

Se utiliza GitHub Projects		x	No implementado.
La estructura del repositorio es organizada	x		Backend y frontend separados.

Sección 2. Estado del Backend

Aspecto	Estado	Observaciones
Proyecto ejecuta correctamente	Correcto	Funciona adecuadamente.
Arquitectura por capas (Controllers, Services, Models, Routes)	Correcto	Controllers, Models, Services y Routes.
ORM implementado correctamente	Correcto	Eloquent implementado.
Relaciones entre entidades	Correcto	Llaves foráneas implementadas.
CRUD completo	Correcto	Todos los módulos principales.
Validaciones	Correcto	Request Validation.
Autenticación JWT	Correcto	Laravel Sanctum.
Roles y permisos	Correcto	Tres roles implementados.
Manejo de errores	Parcial	Puede fortalecerse.
Variables de entorno (.env)	Correcto	Configuradas
Documentación de la API (Postman/Swagger)	Parcial	Postman disponible

Sección 3. Estado del Frontend

Aspecto	Estado	Observaciones
La aplicación inicia	Sí	Sin inconvenientes.
Navegación correcta	Correcta	React Router.
Responsive	Parcial	Algunas vistas requieren ajustes.
Consumo API	Correcto	Axios.
Manejo de errores	Parcial	Puede mejorar la experiencia del usuario.
Autenticación	Correcta	Login con Sanctum.
LocalStorage	Sí	Persistencia del usuario.
Componentes reutilizables	Sí	Tabla y componentes compartidos.
Accesibilidad básica	si	Puede fortalecerse.

Sección 4. Estado de la Aplicación Móvil

Aspecto	Estado	Observaciones
Compila	Sí	Proyecto funcional.
Navegación	Correcto	En desarrollo.
Login	Correcto	Consume la API.
Consumo API	Correcto	Integrado con Laravel.
Persistencia	Parcial	En implementación.
Responsive	Sí	Adaptado para móviles.

Sección 5. Base de Datos

Aspecto	Estado	Observaciones
Modelo Relacional	Correcto	Modelo relacional normalizado.
Normalización	Correcto	Modelo relacional normalizado.
Llaves Primarias	Correcto	Llaves primarias implementadas
Llaves Foráneas	Correcto	Llaves foráneas implementadas
Integridad Referencial	Correcto	sin observaciones
Scripts DDL	Correcto	Script DDLdisponible.
Scripts DML	Correcto	Script DML disponible.
Datos de prueba	Correcto	Datos de prueba coherentes.

Sección 6. Variables de Entorno

Aspecto	Estado
Archivo .env	Sí
Variables documentadas	Parcial
Claves sensibles protegidas	Sí
Configuración correcta	Sí

Sección 7. Documentación

Documento	Existe	Actualizado
RF	Sí	Sí
RNF	Sí	Sí
Casos de Uso	Sí	Sí
BPMN	Sí	Sí
DER	Sí	Sí
Modelo Relacional	Sí	Sí
Diccionario de Datos	Sí	Sí

Historias de Usuario	no	por hacer
Arquitectura	si	no

Sección 8. Manuales

Manual	Existe	Estado
Usuario	no	por hacer
Técnico	no	por hacer
Instalación	no	por hacer
API	no	por hacer

Sección 9. Calidad

Documento	Existe
Plan Maestro	No
Casos de Prueba	No
Registro de Bugs	No
Matriz de Riesgos	No
Matriz de Trazabilidad	No
Dashboard QA	No

Sección 10. Seguridad Básica

Aspecto	Sí	No	Observaciones
Contraseñas cifradas	x		
JWT		x	se utiliza Laravel Sanctum
Roles	x		
Validaciones Backend	x		

Validaciones Frontend	x		
Protección rutas	x		
Variables sensibles protegidas	x		

Sección 11. Arquitectura

Aspecto	Estado
Arquitectura definida	Sí
Diagrama actualizado	no
Separación por capas	Sí
API desacoplada	Sí
Persistencia	Sí
Escalabilidad	Parcial

Sección 12. Hallazgos

Fortalezas

- Arquitectura desacoplada entre frontend y backend.
- Base de datos bien estructurada.
- Control de roles y permisos.
- Documentación funcional completa.
- CRUD de los módulos principales implementados.

Debilidades

- No existe un proceso formal de QA.
- Faltan casos de prueba documentados.
- No se dispone de un registro de incidencias.
- No se han realizado pruebas de rendimiento ni seguridad.

Riesgos

- Posibles errores no detectados antes del despliegue.
- Vulnerabilidades por falta de pruebas especializadas.
- Incremento de deuda técnica.

Recomendaciones

- Elaborar el Plan Maestro de Pruebas.
- Diseñar casos de prueba.
- Crear una matriz de trazabilidad.
- Implementar pruebas de seguridad y rendimiento.
- Llevar un registro formal de defectos.

Sección 13. Calificación del Proyecto

Área	Puntuación
Repositorio GitHub	8/10
Backend	10/10
Frontend	9/10
Mobile	8/10
Base de Datos	10/10
Documentación	7/10
Seguridad	9/10
QA	4/10
Arquitectura	9/10
Total	74/90

Nivel de madurez

- 81–90: Excelente
- 71–80: Bueno
- 61–70: Aceptable
- 51–60: Requiere mejoras
- Menos de 51: Crítico